

Harness e Arquitetura na Prática

Mateus Oliveira · PPGTI1101 — Desenvolvimento de Software com IA · 13/09/2026

Projeto: **Player Modeling Lab** · Etapa 7 — repositório e histórico de commits: github.com/mateus-oliveira/ia-dev-lab/pull/30

Autonomia e guardrail (Etapa 1). Executei a mesma tarefa pequena — um script CLI de avaliação do modelo — sob plan mode (16 s, nenhum arquivo tocado) e sob auto-accept (21 s, código escrito e executado). A diferença bruta de 5 s esconde o que importa: o plan mode exige duas rodadas e entrega 12 linhas de plano para revisar; o auto-accept entrega 97 linhas prontas em uma rodada. A comparação revelou que **o que decide o modo não é o tamanho da tarefa, é o que ela alcança**: a mesma tarefa seria péssima escolha para auto-accept se tocasse o dataset de treino ou uma migração. Como guardrail criei um hook `PreToolUse` que bloqueia qualquer escrita do agente em `src/data/*.csv` — risco específico deste projeto, porque alterar o dataset muda o modelo servido pela API e **nenhum teste falha**. O hook bloqueou de fato tentativas por `Bash` e por `Write`, e bloqueou até o comando com que eu documentava o próprio hook, por citar o caminho protegido perto de `rm`.

TDD e enforcement (Etapa 2). Implementei um segundo modelo de Bartle (Árvore de Decisão) com o endpoint devolvendo as duas previsões, em Red-Green-Refactor com um commit por fase. O passo RED revelou uma tensão real: **o harness de pré-commit impede commitar teste vermelho**, porque `mypy` e `pytest` não distinguem "vermelho porque ainda não implementei" de "vermelho porque quebrei" — usei `--no-verify` de forma deliberada e registrada. Instalei o `td-guard` (`npm + reporter pytest`), que resolve exatamente essa lacuna agindo na edição e não no commit: ele bloqueou a criação de um terceiro modelo sem teste, citando o estado real do `pytest`. Seu ponto cego: não cobre escrita via `Bash`, e a sessão inteira desta atividade editou arquivos por `Bash`. Na comparação com/sem TDD, o resultado honesto é que **a ausência de TDD não produziu bugs** — 10 dos 11 testes escritos depois passaram de primeira. Ela produziu três commits sem rede alguma e uma pergunta de design que nunca me ocorreu enquanto implementava: o que garante que os modelos registrados, os treinados pela API e os campos da resposta continuem os mesmos? O ganho do TDD aqui foi pressão de design e liberdade de refatorar, não correteude imediata.

Checkpoint humano (Etapa 3). Defini que nenhuma alteração no contrato de resposta de um endpoint público é implementada sem aprovação explícita. Escolhi esse ponto porque todos os outros riscos do projeto já têm hook, e este é indecível por máquina. Na simulação, o agente parou e apresentou quatro formatos; **eu não aprovei nenhum — editei**, definindo `{"player_id", "knn", "decision_tree"}`. O papel que assumi foi de revisor com poder sobre contrato e arquitetura, não revisor de diff: não li as ~1.200 linhas linha a linha, decidi o contrato, a arquitetura, o escopo do modelo novo e desbliquei o agente quando um hook mal configurado derrubou a sessão. O limite desse arranjo é que ele depende do harness — onde não há teste nem hook, não há revisão nenhuma.

Decisão arquitetural, ADR e diagrama (Etapas 4 e 5). O levantamento sobre o código real encontrou quatro dependências invertidas (`ml/` → `api/schemas`, `worker/` → `simulator/`, `worker/` e `alembic/` → `api/database`), todas com a mesma causa: não existia camada de domínio. Decidi **não extrair serviço** — checando os critérios um a um, não há escala, times nem ciclos de release independentes, e extrair antes de corrigir transformaria um import ruim em contrato distribuído ruim — e **extrair domain/ e persistence/**, registrado na ADR 0011 e implementado sem alterar uma única asserção de teste. Ganho verificado: `ml/` já não carrega FastAPI. Gerei o diagrama C4 de duas formas; a versão em sintaxe `C4Container` a partir de um prompt genérico ficou ortodoxa mas com a **fronteira errada** (tratava `ml/` como contêiner, sugerindo exatamente o serviço que a ADR rejeita), e a versão refeita com o contexto da análise separa processo de biblioteca e distingue fluxo de dados de dependência de código. Escolhi a segunda: um diagrama que induz a decisão errada é pior que um diagrama feio. O aprendizado foi que a qualidade do diagrama dependeu do contexto, não do modelo.

Dívida técnica (Etapa 6, opção A). Rodei o `ruff` com ruleset completo como auditoria. Entre ~120 apontamentos, quase todos ruído, apareceu `S105`: o segredo de assinatura dos JWT tinha fallback embutido no código — e o `.env.example` distribuía o mesmo valor, de modo que o caminho documentado no README reproduzia o segredo publicado. Demonstrei o impacto: forjei um token só com a chave lida do repositório e obtive **HTTP 200 em /auth/me** para um usuário cuja senha eu nunca soube. Corrigi (ADR 0012). O principal aprendizado: **a dívida não estava em código feio** — estava em código limpo, tipado, documentado, com testes verdes, revisado por humano e aprovado pelo CI. Só apareceu quando uma ferramenta com outro critério olhou para ele.

Dificuldade real enfrentada. Registrei os hooks com caminho relativo (`python3 scripts/...`), copiando o estilo do hook que já existia. Quando um comando mudou o diretório de trabalho da sessão, os dois hooks passaram a falhar com "arquivo não encontrado" — saída diferente de zero, que o Claude Code trata como bloqueio. Como o matcher cobria `Bash|Write|Edit|NotebookEdit`, perdi todas as ferramentas de escrita e execução de uma vez, e **não conseguia sequer corrigir o arquivo que me bloqueava**; foi preciso intervenção manual. A lição foi maior que o bug: o custo de um guardrail não é só o que ele bloqueia de propósito. Um guard-rail mal configurado tem modo de falha próprio, e convém que ele falhe aberto em vez de fechado — foi exatamente o critério que apliquei ao escrever o hook de dados de origem, que permite a execução quando não consegue interpretar o payload.